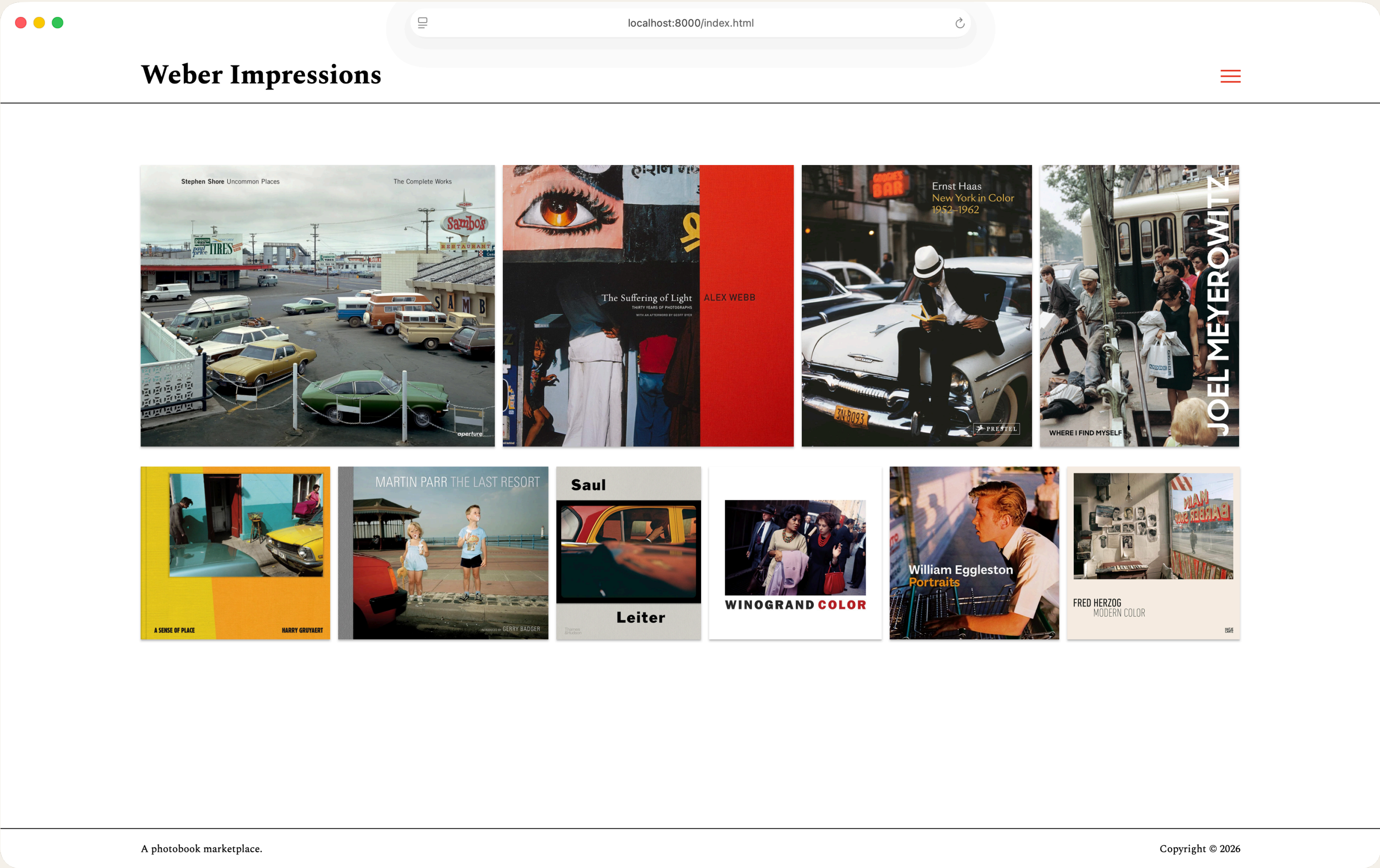


Concierge

Customer service agent for **Weber Impressions**, a fictional online photobook marketplace.

Clone the repo, paste your Anthropic key into `.env`, `python server.py`. Type `aaa` to begin.

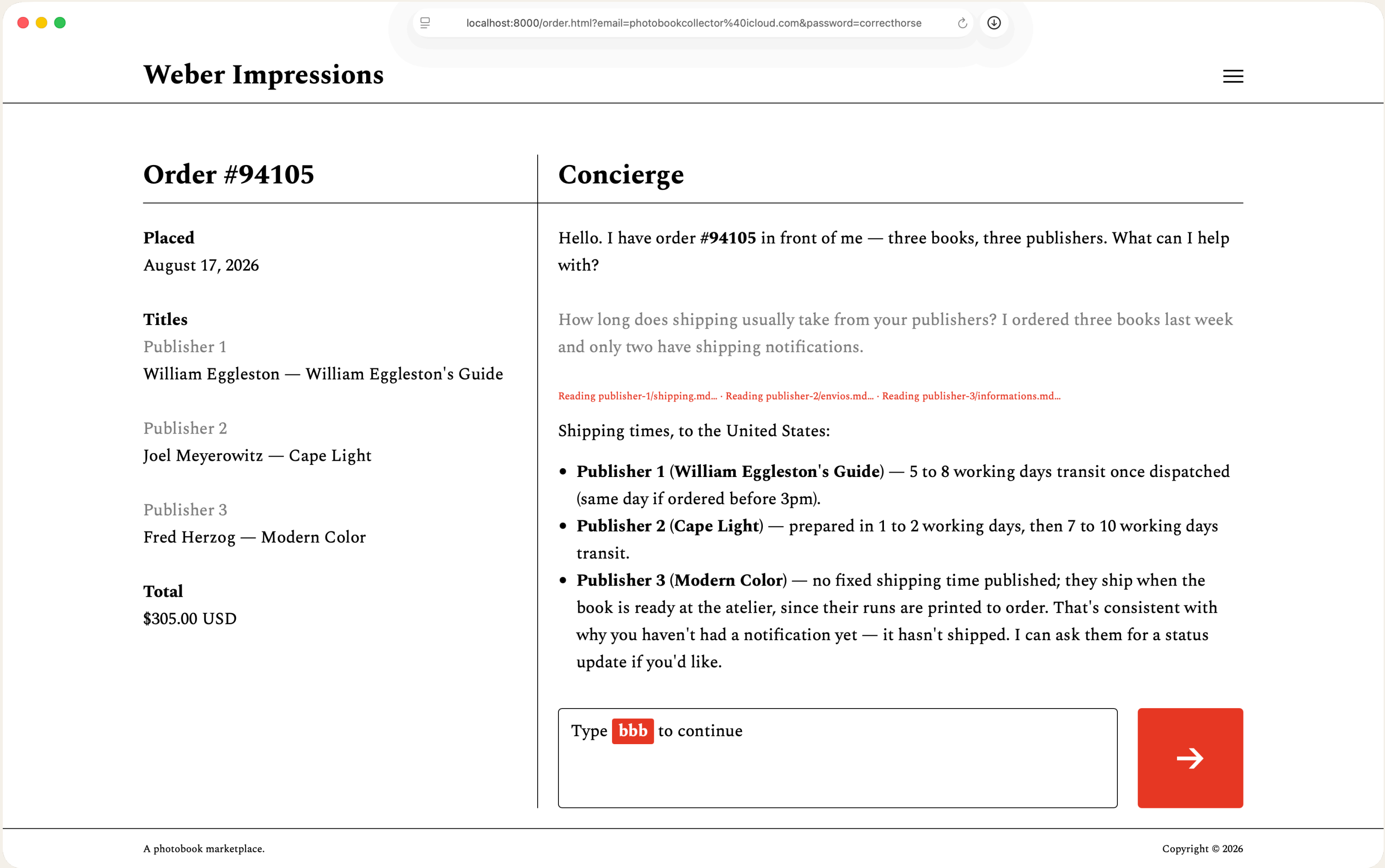


Thesis

What's your core belief about how a great customer service agent should work, and how does your implementation reflect that?

I believe a great customer service agent should:

- Have **authentication** — know who it's talking to before the conversation starts.
- Be **embedded** in the customer experience (e.g., the website), not bolted on.
- Have **context** (e.g., the Order | Concierge page).
- Relay only what a **source** says (e.g., an FAQ, a live order lookup) — never make anything up.
- **Act** on its own when it's routine — and let code, not judgment, make sure the moments that matter reach a person.
- Know when to **escalate** and where — every question has an owner (e.g., a recommendation → the booksellers) — and a conversation with your customer is valuable.



Architecture

How does a customer inquiry flow through your system? Cover the key components: agent orchestration, tools, memory, and prompts.

The browser talks to a small server, and a loop decides each turn (answer, ask, act, or escalate).

Loop — a model call per round; it only goes round again for a tool call.

Tools — read a FAQ, check a publisher, cancel a line, escalate, get a person. One gate in code: cancel only what hasn't been charged.

Memory — the conversation (held by the page) and the order (held by the server).

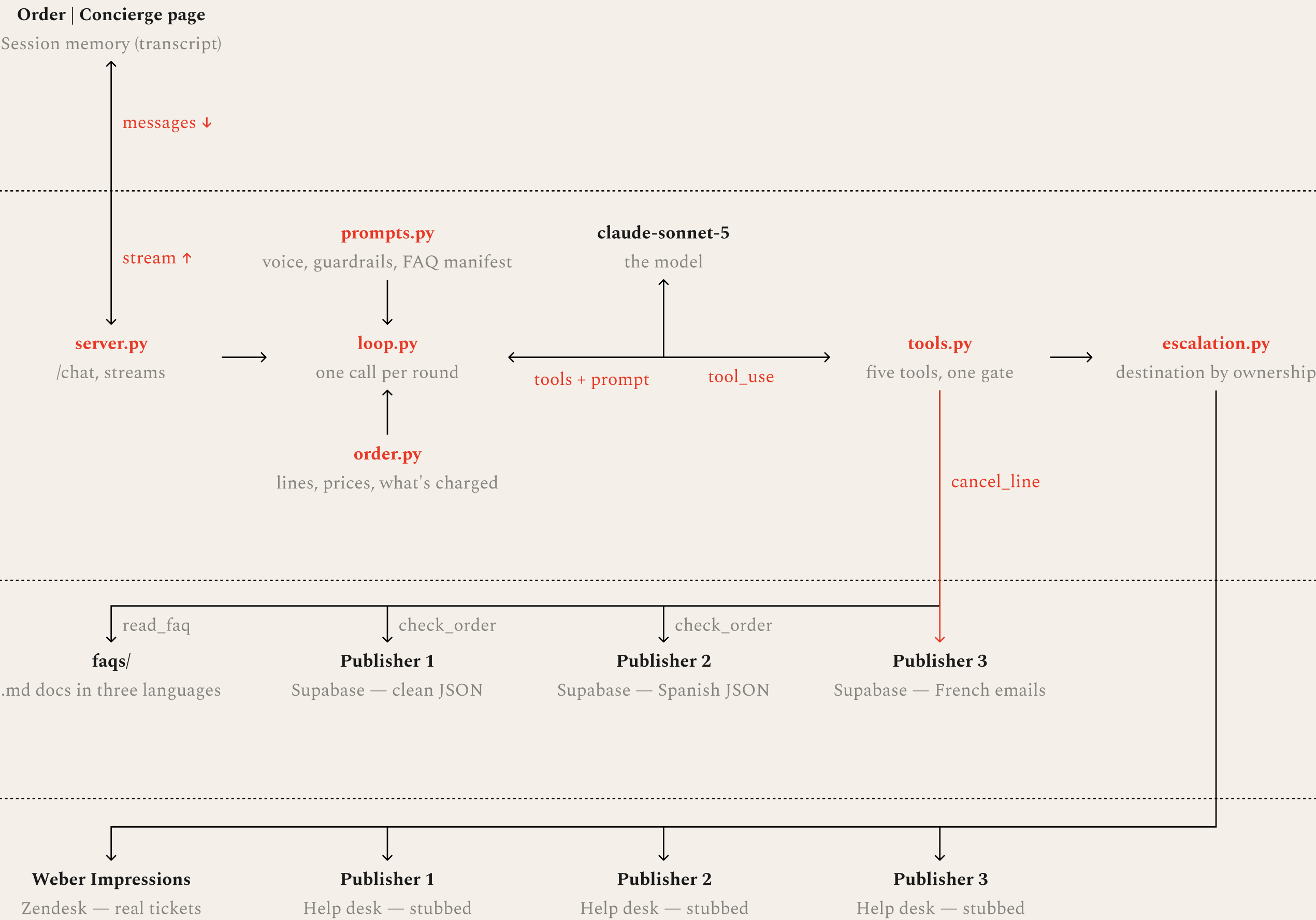
Prompts — the voice, three guardrails, and a manifest of everything the agent may read.

Customer

Concierge

Truths

People



Decisions

Pick 2–3 choices that mattered most (e.g., how you handle tool use, hallucination prevention, guardrails, intent disambiguation). What did you choose, what did you trade off, and why was it worth it?

Routine is automated; moments get a person

Chose
The model decides what to do; code decides what's allowed. A line that hasn't been charged cancels on its own — routine. Everything after shipment goes to a person.

Traded
Automation. The agent resolves less on its own — returns, refunds, exceptions all go to a person. No value thresholds: a \$150 cancel needs no human; a \$5 refund does.

Worth it
Every handoff puts a person in front of the customer — for a shop like this, that conversation is the loyalty.

A reading list, not a search engine

Chose
Hand the agent the entire list of documents — and who wrote each one — and let it read whole files.

Traded
Scale. A company with ten thousand help articles can't do this; they need a search engine.

Worth it
It has a reading list and it reads — and when it reads the wrong document, you can see exactly why.

No sentiment detection

Chose
A safety net: if the conversation stalls — six customer messages, nothing looked up, nothing done — a person is brought in automatically.

Traded
A wording-detector misses precisely the customer who matters — an upset collector writes “nobody has replied,” not “RIDICULOUS.”

Worth it
Every ticket says why it was opened, so nobody at the help desk has to guess.

Change

Given more time or a production context, what's the first thing you'd change and why?

Memory that persists

Built

The conversation lives in the page and dies on reload. One customer, one order, no real sign-in.

Change

Real sign-in. Many orders. Conversations that carry over — and a cancellation that sticks.

Why

Memory is the one part built demo-shaped. The loop, the gate, and the routing go to production as they are.

Voice as a channel

Built

Chat only.

Change

High-spend customers and high-dollar orders are offered voice.

Why

Spend buys a richer way to reach a person — it never changes what the agent may say or do.

Gaps reported

Built

Publisher 3 publishes no shipping time. Concierge says so — gracefully, and silently.

Change

Every “I couldn't source that” becomes a signal: which document was missing, how often, for whom.

Why

In production the missing page is normal — policies change and nobody updates the article. Stale articles, not the model, are the limit.